

- A. Please describe how the Pack parallel pattern works. How is it implemented (pseudo code and PRAM complexity required)? (2)

See slides lecture-10_Patterns-c pp. 56-59.

- B. Consider the following nested loop:

```
for (j = 0; j < 999; j++) {  
    for (i = 2; i < 999; i++) {  
        a[i][j] = f(a[i-2][j]);  
    }  
}
```

Evaluate the cache-friendliness of this code and consider its relationship to parallelizability. How does the memory layout of the array affect performance, and what changes could improve cache utilization vs parallelizability? Justify your answer. Assume that a stores integers. (2)

Original code is fully parallelizable with respect to loop j if function f is safe and it is partially parallelizable with respect to loop i. The original code is poor with respect to cache friendliness because the variable "a" is column-wise accessed. Swapping the two loops fixes cache friendliness. Tiling could be another approach to improve cache friendliness and parallel computation.

- A. What is the impact of memory coalescing on DRAM access efficiency in CUDA? (1) **This question can be skipped if you got 1 or 3 points in the second challenge.**

When all threads of a warp execute a load instruction, if all accessed locations fall into the same burst section (coalesced access), only one DRAM request will be made and the access is fully coalesced.

- B. What is the purpose of using shared memory in tiled matrix multiplication? Is the corner turning technique relevant for this purpose? (2) **This question can be skipped if you got 2 or 3 points in the second challenge.**

Reading A and B in a matrix multiplication results in A read in non-coalesced way and B read in a coalesced way. To improve performance, we copy the data into shared memory, allowing both input matrices to be accessed coalesced in both cases. In addition, the computation working on the shared memory can access the two tiles in any order: "corner turning technique".

See slides lecture-5_gpuarch_2nd pp. 19-23.

- C. Please describe how priority scatter works. (1)

See slides lecture-10_Patterns-c page 53.

- A. Efficient parallel programming with CUDA requires knowledge about GPU architecture. Discuss this claim by providing concrete examples. (2)

Without knowing

- B. What is Unified Memory in CUDA? When is it used? (2)

Unified Memory is an abstraction that lets the programmer treat the host and device memories as a single address space. Data transfers between host and device happen reactively at runtime, triggered by page faults. It is used to simplify the programming (so that data does not need to be moved explicitly between host and device) and it might be useful for sparse/irregular workloads or in multi-GPU systems.

- A. Provide an example (in pseudocode, with explanations) of a batch processing pattern where OpenMP and CUDA are used together to distribute work among multiple GPUs. (2)

```
int deviceCount;

cudaGetDeviceCount(& deviceCount);

std::vector<cudaStream_t> streams(deviceCount);

#pragma omp parallel for num_threads(deviceCount)
    for(int dev = 0; dev < deviceCount; ++dev) {    // for each GPU in the system
        cudaSetDevice(dev);

        cudaStreamCreate(&streams[i]);              // use a stream for data transfers

        // Allocate, initialize and transfer memory
    }

#pragma omp parallel for num_threads(deviceCount)
    for(int dev = 0; dev < deviceCount; ++dev) {
        cudaSetDevice(dev);

        kernel<<<gridDim, blockDim, streams[i]>>>(...);
    }
```

- B. Provide an example (in pseudocode, with explanations) of an application that uses OpenMP instead of CUDA to offload computation on a GPU. (2)

```
#pragma omp target map(a,b,c,d)    // offload the following region to a GPU, transfer contents of a,b,c,d
{
    #pragma parallel for            // use multiple GPU threads to parallelize this loop
    for (i=0; i<N; i++) {
        a[i] = b[i] * c + d;
    }
} // End of target
```